

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

```
# Q1
def pow(a,m):
    if m==0:
        return 1
    elif m==1:
        return a
    else:
        return a*pow(a,m-1)
print(f'2 power of 3 : {pow(2,3)}')
```

2 power of 3 : 8

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

```
# Q2
def gcd(x,y):
    if y==0:
        return x
    else:
        return gcd(y,x%y)
print(f'GCD(12,8) : {gcd(12,8)}')
```

GCD(12,8) : 4

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

```
# Q3
def generate_increasing(n, current="", last_digit=0):
    if len(current) == n:
        print(current, end=" ")
        return
    for digit in range(last_digit + 1, 10):
        generate_increasing(n, current + str(digit), digit)
n = 3
print(f'Increasing numbers of length {n}:')
generate_increasing(n)
```

Increasing numbers of length 3:

123 124 125 126 127 128 129 134 135 136 137 138 139 145 146 147 148 149 156 157 158 159 167 168 169 178 179 189 234 235 236

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

```
# Q4
def fibbo(n):
    if n==0 or n==1:
        return n
    else:
        return fibbo(n-1)+fibbo(n-2)
n = 5
print(f'Fibonacci number at position {n} : {fibbo(n)}')
```

Fibonacci number at position 5 : 5

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

```
# Q5
import random
def kth_largest(arr, k):
    def partition(left, right, pivot_index):
        pivot = arr[pivot_index]
        arr[pivot_index], arr[right] = arr[right], arr[pivot_index]
        store_index = left
        for i in range(left, right):
            if arr[i] > pivot:
                arr[store_index], arr[i] = arr[i], arr[store_index]
                store_index += 1
        arr[store_index], arr[right] = arr[right], arr[store_index]
        return store_index

    def quickselect(left, right, k_smallest):
        if left == right:
            return arr[left]
        pivot_index = random.randint(left, right)
        pivot_index = partition(left, right, pivot_index)
        if k_smallest == pivot_index:
            return arr[k_smallest]
```

```

        elif k_smallest < pivot_index:
            return quickselect(left, pivot_index - 1, k_smallest)
        else:
            return quickselect(pivot_index + 1, right, k_smallest)
    return quickselect(0, len(arr) - 1, k - 1)

arr = [3, 1, 5, 12, 2, 11, 7, 4]
k = 3
print(f"Array: {arr}")
print(f"{k}rd largest element: {kth_largest(arr, k)}")

```

Array: [3, 1, 5, 12, 2, 11, 7, 4]
3rd largest element: 7

''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''

Q6

a) $O(n^2)$ b) $O(n)$ c) $O(2^n)$

```

''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''

# Q7
import math
def has_antipodal_points(points):
    def normalize(angle):
        return angle % (2 * math.pi)
    angles = sorted([normalize(math.atan2(y, x)) for x, y in points])
    n = len(angles)
    j = 0
    for i in range(n):
        opposite = normalize(angles[i] + math.pi)
        while j < n and angles[j] < opposite - 1e-9:
            j += 1
        if j < n and abs(angles[j] - opposite) < 1e-9:
            return True
    return False
points = [(1, 0), (-1, 0), (0, 1)]
print("Has antipodal points:", has_antipodal_points(points))

```

Has antipodal points: True

```

''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''

# Q8
def partition(arr, low, high):
    pivot = arr[low]
    i = low + 1
    j = high
    while True:
        while i <= j and arr[i] <= pivot:
            i += 1
        while i <= j and arr[j] > pivot:
            j -= 1
        if i <= j:
            arr[i], arr[j] = arr[j], arr[i]
        else:
            break
    arr[low], arr[j] = arr[j], arr[low]
    return j
def quick_sort_helper(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quick_sort_helper(arr, low, pi - 1)
        quick_sort_helper(arr, pi + 1, high)
def quick_sort(arr):
    quick_sort_helper(arr, 0, len(arr) - 1)
arr = [37, 2, 6, 4, 89, 8, 10, 12, 68, 45]
quick_sort(arr)
print("Sorted Array:", arr)

```

Sorted Array: [2, 4, 6, 8, 10, 12, 37, 45, 68, 89]

```

''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''

# Q9
def selection_sort(data):
    arr = data[:]

```

```

n = len(arr)
for i in range(n):
    min_idx = i
    for j in range(i + 1, n):
        if arr[j][1] < arr[min_idx][1]:
            min_idx = j
    arr[i], arr[min_idx] = arr[min_idx], arr[i]
return arr
def bubble_sort(data):
    arr = data[:]
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j][1] > arr[j + 1][1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
def insertion_sort(data):
    arr = data[:]
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j][1] > key[1]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr
def to_dict(sorted_data):
    return {name: f"${worth} Billion" for name, worth in sorted_data}
personalities = [
    ("Elon Musk", 240),
    ("Bill Gates", 130),
    ("Jeff Bezos", 180)
]
print("\nSelection Sort:", to_dict(selection_sort(personalities)))
print("Bubble Sort: ", to_dict(bubble_sort(personalities)))
print("Insertion Sort:", to_dict(insertion_sort(personalities)))

```

```

Selection Sort: {'Bill Gates': '$130 Billion', 'Jeff Bezos': '$180 Billion', 'Elon Musk': '$240 Billion'}
Bubble Sort:   {'Bill Gates': '$130 Billion', 'Jeff Bezos': '$180 Billion', 'Elon Musk': '$240 Billion'}
Insertion Sort: {'Bill Gates': '$130 Billion', 'Jeff Bezos': '$180 Billion', 'Elon Musk': '$240 Billion'}

```

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

Q10

```

def merge_sort_strings(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort_strings(arr[:mid])
    right = merge_sort_strings(arr[mid:])
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

```

```

words = ["banana", "apple", "cherry", "date"]
print("Sorted:", merge_sort_strings(words))

```

```
Sorted: ['apple', 'banana', 'cherry', 'date']
```

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

Q11

```

def merge_sorted_lists(list1, list2):
    result = []
    i = j = 0
    while i < len(list1) and j < len(list2):
        if list1[i] <= list2[j]:
            result.append(list1[i])
            i += 1
        else:
            result.append(list2[j])
            j += 1

```

```
result.extend(list1[i:])  
result.extend(list2[j:])  
return result
```

```
list1 = [1, 3, 5, 7]  
list2 = [2, 4, 6, 8]
```

```
print("Merged sorted list:", merge_sorted_lists(list1, list2))
```

```
Merged sorted list: [1, 2, 3, 4, 5, 6, 7, 8]
```